1 Introduction

Raft is a consensus algorithm designed for understandability and ease of implementation compared to the Paxos algorithm, and thus, it is widely utilized in various products to resolve consensus issues.

In distributed systems, Raft requires odd number of servers to always maintain a quorum that remains functional and communicative in case of network partition faults. Practically, this means a Raft cluster size should be at least three to tolerate a single point of failure.

While the requirement for an odd cluster size isn't problematic for large-scale distributed systems, it can pose a challenge for budget-constrained customers who need to reach consensus with fewer servers. One solution to this issue is to substitute one server in the cluster with a cost-effective entity known as a witness. The witness acts as a tie-breaker, ensuring the maintenance of a quorum in case of an even split of servers due to network issues. Witness typically operates on low-configuration hardware to save costs, seldom participates in critical data paths, and only persists a minimal amount of data. In the Paxos algorithm, a witness can be implemented as shared storage, which is not only inexpensive but also widely available to most customers.

Efforts have been undertaken to incorporate witness into Raft algorithm. For example, Diego Ongaro's Ph.D. dissertation explored the reduction of server numbers by using a witness to store log entries for any failed server until recovery or replacement. Jehan-François Pâris and Darrell D. E. Long proposed a type of follower-only server witness that doesn't persist user data. TiKV implements a witness in a similar manner as described by Ongaro: a log-only entity that persists only raft logs without applying them, and supports switching between witness and non-witness servers.

However, all existing research and implementations necessitate a standalone server as a witness, adding to deployment complexity and reducing cost-efficiency compared to shared storage in the Paxos algorithm. Additionally, the witness must participate in log replication when it needs to conform to a quorum with other servers (when some servers are down), implying that its low-configuration hardware could potentially become a performance bottleneck in such scenarios.

2 Algorithm

In this section, we present the extended Raft algorithm, which is a variant of the Raft algorithm. It is designed for cluster with regular servers and at most one witness. The witness functions strictly as a follower server, implying that its role never transitions to candidate or leader. Furthermore, a witness only maintains a minimal set of metadata.

Our newly proposed algorithm minimizes data traffic and reduces the frequency of witness visits while preserving all key properties of the Raft algorithm. This implies that the extended Raft algorithm ensures that each of the following properties is consistently upheld.

- Election Safety
- Leader Append-only
- Log Matching
- Leader Completeness
- State Machine Safety

In the rest of this document, unless otherwise specified, the term 'server' will denote either a regular server or a witness. Consequently, the server set, denoted as *Server*, that constitutes a cluster could either be $RegularServer \cup witness$ or simply *RegularServer*, where *RegularServer* is the set of all regular servers.

2.1 Concepts and Definitions

Definition 1. *A leader's* ***replication set*** *is a subset of the cluster server set, with its cardinality matching that of the regular server set.*

Let *ReplicationSets* represent the set of all replication sets within a cluster. We have:

$$ReplicationSets \triangleq \begin{cases} \{Server\}, & witness \notin Server \\ \{Server \setminus \{x\} : x \in Server\}, & witness \in Server \end{cases}$$

A leader maintains its replication set based on its view of the servers and modifies it under specific conditions. A leader may alter its replication set multiple times within a single term. To uniquely identify each replication set, we introduce the notion of a 'subterm' as detailed below.

Definition 2. *A term is segmented into* SUBTERMS*, each begins with a replication set.*

Subterms are sequentially numbered using consecutive integers, beginning from 0 for each term. The leader retains its current subterm and increments it when the replication set is altered. Consequently, a leader's replication set remains static throughout its subterm.

2.2 States

Figure 2.1 illustrates the additional states introduced by the Extended Raft algorithm to accommodate the witness.

Each leader maintains three additional volatile variables: *replicationSet*, *currentSubterm*, and *witnessSubterm*. The *replicationSet* represents the leader's current replication set, initialized to *RegularServer*, and is modified by *AdjustReplicationSet* action. The *currentSubterm* represents the latest subterm of the leader, initialized to 0 and incremented upon a change in the leader's replication set. The *witnessSubterm* denotes the latest subterm during which the leader replicated its log entry to the witness.

In addition to *currentTerm* and *votedFor*, the witness also includes *witnessReplicationSet*, *witnessLastLogTerm*, and *witnessLastLogSubterm*. These represent the most recent replication set, log entry term, and log entry subterm that the leader sent to the witness. We will further discuss this in Section 2.3.

Beyond the additional states in the leader and witness, each log entry is also associated with the leader's current subterm number when it is appended to the leader's log (Figure 2.2). This subterm number is replicated and persisted alongside the log entry on all regular servers. We now denote a log entry as $\langle index, term, subterm \rangle$, which is uniquely identified by *index* and *term*, and associated with *subterm*.

2.3 Log Replication

The leader replicates its log to regular servers in the exact same way as that in the Raft algorithm, and the regular servers handle the received log entries in the same way as well.

However, for the witness, log replication is performed differently, as shown in Figure **??**. In the Extended Raft algorithm, the leader sends an *AppendEntriesToWitnessRequest*

```
The following variables are used only on leaders:
The next entry to send to each follower.
VARIABLE nextIndex
The latest entry that each follower has acknowledged is the same as the
leader's. This is used to calculate commitIndex on the leader.
VARIABLE matchIndex
Leader's replication set
VARIABLE replicationSet
Leader's subterm number
VARIABLE currentSubterm
The latest subterm that this leader has written to the witness
VARIABLE witnessSubterm
leaderVars ≜ ⟨nextIndex, matchIndex, elections,
replicationSet, currentSubterm, witnessSubterm⟩

The set of log entries that are acknowledged by witness, e.g. not shortcut replication.
This is used to assist invariant and does not exist in real implementation.
VARIABLE witnessAcks

The following variables are persisted only on witness:
The latest replication set sent from leader
VARIABLE witnessReplicationSet
The latest term of replicated log entry
VARIABLE witnessLastLogTerm
The latest subterm of replicated log entry
VARIABLE witnessLastLogSubterm
witnessVars ≜ ⟨witnessReplicationSet,
witnessLastLogTerm, witnessLastLogSubterm⟩
```

Figure 2.1: State

```
Leader i receives a client request to add v to the log.
ClientRequest(i, v) ≜
    ∧ state[i] = Leader
    ∧ LET entry  ≜ [term        ↦ currentTerm[i],
                    subterm     ↦ currentSubterm[i],
                    value       ↦ v]
                   newLog ≜ Append(log[i], entry)
                   IN  log' = [log EXCEPT ![i] = newLog]
    ∧ UNCHANGED ⟨messages, serverVars, candidateVars,
                 leaderVars, commitIndex, witnessVars⟩
```

Figure 2.2: Client request

to the witness, which includes the current replication and metadata of the log entry that satisfies the following conditions:

1. The log entry's term and subterm are equal to *currentTerm* and *currentSubterm*, respectively.

2. The leader's current replication set includes the witness.

3. The leader has received acknowledgments for the log entry from at least a subquorum (one server away from forming a quorum) in the leader's current replication set.

4. The leader has not received any acknowledgment from the witness during the current subterm.

The *AppendEntriesToWitness* can be considered as a special implementation of the *AppendEntries* action with batching of log prefix up to a specific log entry. This log entry belongs to the current subterm and has received subquorum acknowledgements from the current replication set. In addition to the message fields that also exist in *AppendEntriesRequest*, the dispatched message *AppendEntriesToWitnessReques* also contains the current replication set and the metadata of the last batched entries.

Upon receiving the request, the witness persists the replication set, as well as the term and subterm of the log entry, then responds to the leader with an

```
Leader i sends witness an AppendEntriesToWitness request with log entry
in current subterm acknowledged by a subquorum in current replication set.
AppendEntriesToWitness(i) ≜
    ∧ state[i] = Leader
    ∧ WitnessServer ∈ replicationSet[i]
    ∧ LET Agree(index) ≜
        {i} ∪ {k ∈ replicationSet[i] : matchIndex[i][k] ≥ index}
      IsAgreed(k) ≜
        ∧ log[i][k].term    = currentTerm[i]
        ∧ log[i][k].subterm = currentSubterm[i]
        ∧ ({WitnessServer} ∪ Agree(k)) ∈ Quorum
      agreeIndexes ≜ {k ∈ 1 .. Len(log[i]) : IsAgreed(k)}
      lastEntry ≜ Max(agreeIndexes)
    IN  ∧ agreeIndexes ≠ {}
        ∧ ∨ witnessSubterm never exceeds currentSubterm following specification
          ∧ currentSubterm[i] > witnessSubterm[i]
          ∧ Send([mtype          ↦ AppendEntriesToWitnessRequest,
                  mterm          ↦ currentTerm[i],
                  mlogTerm       ↦ log[i][lastEntry].term,
                  mlogSubterm    ↦ log[i][lastEntry].subterm,
                  mreplicationSet ↦ replicationSet[i],
                  mindex         ↦ lastEntry,
                  mlog and mentries are used as history variable for
                  the proof. They do not exist in a real implementation.
                  mlog           ↦ log[i],
                  mentries       ↦ SubSeq(log[i], 1, lastEntry),
                  msource        ↦ i,
                  mdest          ↦ WitnessServer])
          ∨ shortcut replication
          ∧ currentSubterm[i] = witnessSubterm[i]
          ∧ Send([mtype          ↦ AppendEntriesResponse,
                  mterm          ↦ currentTerm[i],
                  msuccess       ↦ TRUE,
                  mmatchIndex    ↦ lastEntry,
                  msource        ↦ WitnessServer,
                  mdest          ↦ i,
                  mshortcut      ↦ TRUE])
        ∧ UNCHANGED ⟨serverVars, candidateVars, leaderVars, logVars, witnessVars⟩
```


Figure 2.3: AppendEntriesToWitness

AppendEntriesResponse message (Figure 2.4).

Note that the *mlog* and *mentries* fields in the message are solely used for proof. They do not exist in the actual implementation, nor will the witness persist the full log prefix in its durable storage. The persistence of *m.mentries* to *log* in *HandleAppendEntriesToWitnessRequest* is exclusively used for proof purposes.

When the leader receives an acknowledgment from the witness, it also updates its *witnessSubterm* to the current subterm if the acknowledged log entry is in the current subterm. Then, according to condition **??**, the leader no longer sends *AppendEntriesToWitnessRequest* in the current subterm. Instead, the leader treats subsequent log entries as already acknowledged by the witness if they fulfill all conditions except condition **??**. This is referred to as **Shortcut Replication**, where the leader sends an *AppendEntriesResponse* to itself on behalf of the witness.

Note that the log entry in *AppendEntriesToWitness* must be within the current term and subterm. So, when the leader advances its subterm (a consequence of replication set adjustment), a new empty log entry must be appended to the leader's log. This is to ensure the leader won't be impacted by condition 1 in committing log entries.

Leader immediately commits a log entry during its term and subterm when the log entry is acknowledged by a quorum, in the same way as in Raft. The acknowledgment either comes from a regular server or the witness. Since the leader sends *AppendEntriesToWitnessRequest* to the witness only after it has received subquorum acknowledgments from regular servers, any log entry acknowledged by the witness is immediately committed.

2.4 Replication Set Adjustment

Leader maintains the replication set and modifies it whenever necessary. This process is referred to as **Replication Set Adjustment**. Figure 2.5 illustrates a straightforward method to adjust the replication set. Although there could be multiple ways to adjust a replication set in practical implementations, such a simple method can be used in the algorithm to minimize the atomic regions without loss of generality.

To commit, the leader's replication set must contain at least a subquorum of healthy regular servers if it includes a witness. Therefore, in a practical implementation, replication set adjustment should be performed in a way such

Witness receives an AppendEntriesToWitnessRequest from server j with m.mterm ¡= currentTerm[WitnessServer].

$HandleAppendEntriesToWitnessRequest(j,\ m) \triangleq$

$\land\ m.mterm \leq currentTerm[WitnessServer]$

$\land\ \lor$ reject request

$\land\ m.mterm < currentTerm[WitnessServer]$

$\land\ Reply([mtype \mapsto AppendEntriesResponse,$

$mterm \mapsto currentTerm[WitnessServer],$

$msuccess \mapsto \text{FALSE},$

$mmatchIndex \mapsto 0,$

$msource \mapsto WitnessServer,$

$mdest \mapsto j],$

$m)$

$\land\ \text{UNCHANGED}\ \langle serverVars,\ logVars,\ witnessVars\rangle$

$\lor$ accept request, always no conflict.

$\land\ m.mterm = currentTerm[WitnessServer]$

$\land\ \lor\ m.mlogTerm > witnessLastLogTerm$

$\lor\ \land\ m.mlogTerm = witnessLastLogTerm$

$\land\ m.mlogSubterm > witnessLastLogSubterm$

$\land\ witnessReplicationSet' = m.mreplicationSet$

$\land\ witnessLastLogTerm' = m.mlogTerm$

$\land\ witnessLastLogSubterm' = m.mlogSubterm$

log will not be modified in real implementation. It is only used here for the proof.

$\land\ log' = [log\ \text{EXCEPT}\ ![WitnessServer] = m.mentries]$

$\land\ Reply([mtype \mapsto AppendEntriesResponse,$

$mterm \mapsto currentTerm[WitnessServer],$

$msuccess \mapsto \text{TRUE},$

$mmatchIndex \mapsto m.mindex,$

$msource \mapsto WitnessServer,$

$mdest \mapsto j],$

$m)$

$\land\ \text{UNCHANGED}\ \langle serverVars,\ commitIndex\rangle$

$\land\ \text{UNCHANGED}\ \langle candidateVars,\ leaderVars\rangle$

Figure 2.4: Handle AppendEntriesToWitness

```
Adjust replication set on leader i. This action updates leader's replication
set by a new one.
AdjustReplicationSet(i, s) ≜
    ∧ state[i] = Leader
    ∧ replicationSet[i] ≠ s
    ∧ replicationSet' = [replicationSet EXCEPT ![i] = s]
    ∧ currentSubterm' = [currentSubterm EXCEPT ![i] = @ + 1]
    ∧ UNCHANGED ⟨messages, serverVars, candidateVars, nextIndex, matchIndex,
    elections, witnessSubterm, witnessAcks, log, commitIndex,
    witnessVars⟩
```

Figure 2.5: Replicaiton set adjustment

that the resulting replication set includes as many healthy regular servers as possible. To achieve this, leader can track the status of each peer regular server by monitoring their responses. If leader receives no response from a regular peer server over an *electiontimeout*, it assumes the regular server is unreachable and initiates a replication set adjustment. Let *Reachable* and *Unreachable* represent the set of reachable regular servers and unreachable regular servers from the leader's perspective. A practical replication set adjustment can be implemented as below:

1. When a regular server becomes a leader, its replication set is initialized to all regular servers *RegularServer* in the leader's configuration.

2. If all servers in *RegularServer* are reachable from the leader, leader changes its replication set to *RegularServer*, i.e.,

$$\forall s \in RegularServer : s \in Reachable \implies ReplicationSet' = RegularServer$$

3. Leader swaps one unreachable regular server inside its replication set with

```
Candidate i sends witness a RequestWitnessVote request.
RequestWitnessVote(i) ≜
    ∧ state[i] = Candidate
    ∧ WitnessServer ∉ votesResponded[i]
    ∧ (votesGranted[i] ∪ {WitnessServer}) ∈ Quorum
    ∧ Send([mtype          ↦ RequestWitnessVoteRequest,
    mterm                  ↦ currentTerm[i],
    mlastLogTerm           ↦ LastTerm(log[i]),
    mlastLogSubterm        ↦ LastSubterm(log[i]),
    mvotesGranted          ↦ votesGranted[i],
    msource                ↦ i,
    mdest                  ↦ WitnessServer])
    ∧ UNCHANGED ⟨serverVars, candidateVars, leaderVars, logVars,
    witnessVars⟩
```

Figure 2.6: RequestWitnessVote

the reachable regular server or witness outside, i.e.,

$$\exists x \in \mathit{ReplicationSet}, y \in \mathit{Server} \setminus \mathit{ReplicationSet} : \begin{array}{r} \wedge\ x \in \mathit{Unreachable} \\ \wedge\ \vee\ y \in \mathit{Reachable} \\ \vee\ y = \mathit{witness} \end{array}$$
$$\implies \mathit{ReplicationSet}' = \mathit{Server} \setminus \{x\}$$

4. Leader increments its *currentSubterm* and appends a new empty log entry if its replication set changes.

2.5 Leader Election

When a regular server transitions into candidate, it requests votes from all peer servers including witness. The extended Raft algorithm adheres to the exact same rules as those in Raft when it comes to requesting and granting votes between a candidate and a regular server. However, the candidate does not request a vote from the witness until it has received subquorum votes from regular servers. Figure 2.6 describes the new *RequestWitnessVote* action for a candidate to request vote from a witness.

Considering that the witness does not persist the full log prefix, the extended

Raft algorithm employs a new set of rules for the witness in leader elections. Figure 2.7 describes the new action for the witness to handle a vote request from a candidate. The *RequestWitnessVoteRequest* message includes the *term* and *subterm* of the last log entry in the candidate, as well as the votes the candidate has already received in the current election. The witness votes for a candidate if it has not casted vote in the current term and any of the following conditions is true:

- Candidate's last log entry has larger term, i.e. $m.mlastLogTerm > witnessLastLogTerm$.
- Candidate's last log entry has same term but larger subterm, i.e. $m.mlastLogTerm = witnessLastLogTerm \wedge m.lastLogSubterm > witnessLastLogSubterm$.
- Candidate's last log entry has same term and subterm comparing to witness, and all subqorum votes it got are from regular servers in witness's replication set, i.e.

$$m.mlastLogTerm = witnessLastLogTerm$$
$$m.mlastLogSubterm = witnessLastLogSubterm$$
$$m.mvotesGranted \subseteq witnessReplicationSet$$

2.6 Membership Reconfiguration

Cluster configuration change can be done in both simple and complex approach in Raft algorithm. The simple approach changes one server at a time. And the complex approach allows arbitrary configuration changes at one time.

The key to the safety of configuration change in Raft is preserving overlap between any majority of the old cluster and any majority of the new cluster. This overlap prevents the cluster from splitting into two independent majorities where two leaders being elected in same term, since otherwise the overlapped server would be able to vote two servers in a term, which is a a contradiction of the specification. This applies to the extended Raft algorithm too because a new leader also needs quorum voters to succeed in an election.Therefore, it is safe to apply same membership configuration methods in Raft algorithm to the extended Raft algorithm either with a sequence of single-server membership change or with an intermediate joint configuration that overlaps both old and new membership configurations.

```
Witness receives a RequestWitnessVote request from server j with
m.mterm ¡= currentTerm[WitnessServer].
HandleRequestWitnessVoteRequest(j, m) ≜
    LET logOk ≜  ∨ m.mlastLogTerm > witnessLastLogTerm
     ∨ ∧ m.mlastLogTerm = witnessLastLogTerm
     ∧ m.mlastLogSubterm > witnessLastLogSubterm
     ∨ ∧ m.mlastLogTerm = witnessLastLogTerm
     ∧ m.mlastLogSubterm = witnessLastLogSubterm
     ∧ m.mvotesGranted ⊆ witnessReplicationSet
    grant ≜  ∧ m.mterm = currentTerm[WitnessServer]
     ∧ logOk
     ∧ votedFor[WitnessServer] ∈ {Nil, j}
    IN    ∧ m.mterm ≤ currentTerm[WitnessServer]
     ∧ ∨ grant  ∧ votedFor' = [votedFor EXCEPT ![WitnessServer] = j]
     ∨ ¬grant  ∧ UNCHANGED votedFor
     ∧ Reply([mtype          ↦ RequestVoteResponse,
     mterm          ↦ currentTerm[WitnessServer],
     mvoteGranted          ↦ grant,
      mlog is used just for the 'elections' history variable for
      the proof. It would not exist in a real implementation.
     mlog          ↦ log[WitnessServer],
     msource          ↦ WitnessServer,
     mdest          ↦ j],
     m)
     ∧ UNCHANGED ⟨state, currentTerm, candidateVars, leaderVars,
     logVars, witnessVars⟩
```

Figure 2.7: HandleRequestVoteToWitness

3 Witness Implementation

With the extended Raft algorithm, the witness possesses the following properties that make it particularly suitable for implementation as a storage object:

- Witness does not have volatile variables, implying that all states within the witness are persisted in durable storage.
- Witness only persists a small amount of metadata. Neither user data nor the full log prefix metadata are stored in witness. This results in a trivial and predictable capacity requirement.
- Witness is rarely accessed. The shortcut replication ensures that leader only needs to write to witness when its replication set changes. Therefore, witness may only need to be accessed when faults change (e.g. a server shuts down or recovers) or a candidate requests its vote. In a relatively stable cluster, neither of these events occur frequently.

The absence of volatile variables allows the synchronization of witness states to be distributed across all regular servers. For each log replication between leader and witness, the *HandleAppendEntriesToWitnessRequest* action can be implemented in the leader as a single operation that loads states from the witness's durable storage, make new states according to *HandleAppendEntriesToWitnessRequest*, and writes back new states to the witness's durable storage. Similarly for voting between candidate and witness, the *HandleRequestWitnessVoteRequest* action can be implemented in the candidate as a single operation that loads states from the witness's durable storage, make new states according to *HandleRequestVoteToWitness*, and writes back the new states to the witness's durable storage. These two implementations can be synchronized using an optimistic concurrency control scheme since they both occur infrequently.

Figure 3.1 illustrates an implementation using a share (NFS or SMB) as a witness. We associate a version number with the states, starting from 0. Each modification to the states increases the version by 1. We can then persist each version of the witness states in a file uniquely named after the version number (e.g. <version>.st). Whenever *HandleAppendEntriesToWitnessRequest* or *HandleRequestVoteToWitness* are invoked, a regular server (leader or candidate) loads the witness states from the file with the largest version number, calculates the new states, and saves the new states back to a distinct temporary file (e.g. <server id>.<version>.st). Then, the regular server tries to create a hard link

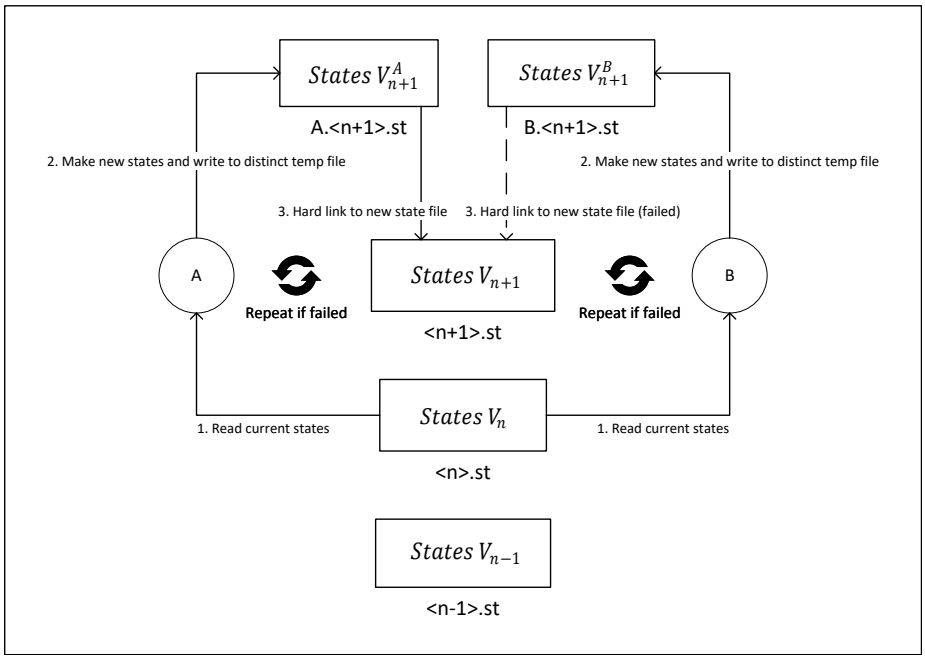


Figure 3.1: Implement witness as a share

from the temporary file to the desired versioned state file (i.e. <version+1>.st). As long as the file system guarantees atomicity in creating hardlinked files, the regular server will only succeed if the target versioned state file does not exist. If any other server has finished update states in the same version, the process needs to be repeated from the beginning. The optimistic concurrency control eliminates the need for locking overhead, which is challenging to implement in practical applications. The infrequency of witness visits makes such a scheme very suitable.

In addition to share, the witness functionality can also be incorporated into cloud storage, such as Azure Storage or AWS S3. Many cloud storage vendors provide optimistic concurrency mechanism for updating object data. For instance, Azure Blob Storage allows clients to update a blob object using the original ETag, combined with a conditional header, to ensure that updates only occur if the ETag remains unchanged. This guarantees that no other client has updated the blob object concurrently. The infrequent access requirements and trivial payload size for witness make cloud storage an ideal solution for this function. Consequently, it provides a broader range of implementation options

for end-users.

4 Discussion

4.1 Compatibility

In a cluster without witness, the extended Raft algorithm operates identically to the Raft algorithm. A cluste can be formed with mixed servers running with either Raft or the extended Raft algorithms. This compatibility facilitates a seamless upgrade from a cluster based on the Raft algorithm to one using the extended Raft algorithm in practical applications. A binary upgrade can be performed in the existing cluster to incorporate the extended Raft algorithm. Following this, the upgraded cluster functions similarly to a typical Raft-based cluster, without the inclusion of a witness. Once all servers have been upgraded to support the extended Raft algorithm, a membership reconfiguration operation can be performed to integrate a witness into the cluster. Conversely, a cluster functioning with the extended Raft algorithm can be reverted back to Raft algorithm through a binary downgrade, which is possible after removal of witness from the cluster.

The compatibility is crucial for real-world applications. It eliminates the necessity to establish a new cluster and migrate data from the existing cluster to take advantage of the benefits offered by a witness. As a result, this minimizes disruptions and enhances the operational efficiency of these applications.

4.2 Availability

The Raft algorithm ensures the presence of a leader that preserves all committed log entries and continues to commit incoming requests, provided that the cluster has a quorum of connected, healthy servers. This implies that if a follower fails, the leader cannot commit if the remaining followers cannot form a quorum with the leader. Additionally, if a leader fails, a new leader can be elected among the other servers that form a quorum.

However, this is a slightly different in the extended Raft algorithm.

4.3 Cascading Failures

The extended Raft algorithm's correctness doesn't guarantee that a leader can always be elected. This predicament often arises when faults occur, leading to

a situation where no regular server has a more up-to-date log than witness. In other words, witness would have been the only feasible next leader if it weren't for the lack of a full log prefix. This typically happens as a result of cascading failures where servers go down sequentially.

For instance, consider a cluster composed of two servers (A and B) and a witness. If A is the leader and a sequence of events unfold as described below, the cluster will find itself in a situation where A is the only functioning regular server, but its log doesn't contain the new committed log entry (made by B). The state in the witness prevents it from casting a vote for A, leaving the cluster in a situation without a leader. However, this doesn't violate the correctness of the extended Raft algorithm. And a new leader can always be elected once B restarts.

1. A is shut down
2. B becomes leader
3. B commits a new entry (hence writes states to witness)
4. A starts up
5. B is shut down before it replicating the new committed entry to A

In real-world scenarios, when a server shuts down, its subsystems do not cease functioning in the same time. For instance, NIC connecting to the witness (NIC_w) may stop after the other NIC that connects to regular servers (NIC_s). Moreover, the server's OS ceases operation after all NICs go down. Considering the same cluster as before, if A was operating as leader in the extended Raft algorithm, unfunctional NIC_s would prompt it to adjust its replication set and advance the subterm. A new log entry may then be committed in the new subterm, with its term and subterm recorded to the witness through the still-functioning NIC_w. Now after A fully shuts down, B won't be elected as the new leader, since the witness, containing more up-to-date states, rejects casting a vote for it.

This particular issue arises due to the limited role of the witness. However, it's a necessary trade-off to minimize the overall footprint and maintain reasonable performance when faults occur. It's important to note that this issue arises in very rare cases. Adding an extra delay in *HandleAppendEntriesToWitnessRequest* can greately reduce the chance of the occurence. The impact to performance is trivial, as most commits do not involve writing to the witness.

4.4 Catching up upon replication set adjustment

When a follower server's log significantly lags behind the leader's, it may take a considerable amount of time for the follower server to catch up. Until then, new log entries in the leader will not be acknowledged by that follower server. If the live servers merely satisfy the minimal quorum number, the leader won't be able to commit until all servers catch up. This situation arises in Raft when a new server is added to a cluster, and the solution is to add the new server as a learner that receives the log but is not a part of the current membership configuration.

This situation can also occur in the extended Raft algorithm when subterm changes. When a server outside replication set recovers from fault state, the leader may rejoin it to the replication set. If there is only a quorum of live servers (including the witness) in the current replication set, the leader won't be able to commit new log entries until all servers catch up. For instance, in the cluster above, if A commits many entries during B's downtime, B's log will significantly lag behind A. After B recovers, A will advance its subterm and change its replication set to $\{A, B\}$. Now, any new entry in A will not be committed until it receives 2 acknowledgements, i.e., both A and B must acknowledge. As a result, the cluster won't be able to commit any entry because it will take B a long time to catch up with A and acknowledge.

The solution to this issue is similar to the learner approach: leader does not change its replication set to add a regular server until that server's log catches up with the leader.

5 Specification

MODULE *extendedraft*

This is the formal specification for the extended Raft consensus algorithm.

< *TODO*: Copyright *Info* >

Copyright 2014 *Diego Ongaro*.

This work is licensed under the Creative Commons Attribution – 4.0 International License https://*creativecommons.org*/licenses/by/4.0/

EXTENDS *Naturals*, *FiniteSets*, *Sequences*, *TLC*, *Bags*, *SequencesExt*

The set of server *IDs*

CONSTANTS *Server*

The set of requests that can go into the *log*

CONSTANTS *Value*

ID of the witness

CONSTANTS *WitnessServer*

Server states.

CONSTANTS *Follower*, *Candidate*, *Leader*

A reserved value.

CONSTANTS *Nil*

Message types:

CONSTANTS *RequestVoteRequest*, *RequestVoteResponse*,
AppendEntriesRequest, *AppendEntriesResponse*,
RequestWitnessVoteRequest, *AppendEntriesToWitnessRequest*

Global variables

A bag of records representing requests and responses sent from one server
to another.

VARIABLE *messages*

A history variable used in the proof. This would not be present in an
implementation.
Keeps track of successful elections, including the initial logs of the
leader and voters' logs. Set of functions containing various things about
successful elections (see *BecomeLeader*).

VARIABLE *elections*

The set of *log* entries that are acknowledged by witness, *e.g.* not shortcut replication.
This is used to assist invariant and does not exist in real implementation.

VARIABLE *witnessAcks*

The following variables are all per server (functions with domain *Server*).

The server's term number.

VARIABLE *currentTerm*

The server's state (Follower, *Candidate*, or *Leader*).

VARIABLE $state$

The candidate the server voted for in its current term, or
Nil if it hasn't voted for any.

VARIABLE $votedFor$

$serverVars \triangleq \langle currentTerm, state, votedFor \rangle$

A Sequence of *log* entries. The index into this sequence is the index of the *log* entry. Unfortunately, the Sequence module defines $Head(s)$ as the entry with index 1, so be careful not to use that!

VARIABLE log

The index of the latest entry in the *log* the state machine may apply.

VARIABLE $commitIndex$

$logVars \triangleq \langle log, commitIndex \rangle$

The following variables are used only on candidates:

The set of servers from which the candidate has received a *RequestVote* response in its *currentTerm*.

VARIABLE $votesResponded$

The set of servers from which the candidate has received a vote in its *currentTerm*.

VARIABLE $votesGranted$

A history variable used in the proof. This would not be present in an implementation.

Function from each server that voted for this candidate in its *currentTerm* to that voter's *log*.

VARIABLE $voterLog$

$candidateVars \triangleq \langle votesResponded, votesGranted, voterLog \rangle$

The following variables are used only on leaders:

The next entry to send to each follower.

VARIABLE $nextIndex$

The latest entry that each follower has acknowledged is the same as the leader's. This is used to calculate *commitIndex* on the leader.

VARIABLE $matchIndex$

Leader's replication set

VARIABLE $replicationSet$

Leader's *subterm* number

VARIABLE $currentSubterm$

The latest *subterm* that this leader has written to the witness

VARIABLE $witnessSubterm$

$leaderVars \triangleq \langle nextIndex,\ matchIndex,\ elections,$
$replicationSet,\ currentSubterm,\ witnessSubterm,\ witnessAcks\rangle$

The following variables are persisted only on witness:

The latest replication set sent from leader

VARIABLE $witnessReplicationSet$

The latest term of replicated *log* entry

VARIABLE $witnessLastLogTerm$

The latest *subterm* of replicated *log* entry

VARIABLE $witnessLastLogSubterm$

$witnessVars \triangleq \langle witnessReplicationSet,$
$witnessLastLogTerm,\ witnessLastLogSubterm\rangle$

End of per server variables.

All variables; used for stuttering (asserting state hasn't changed).

$vars \triangleq \langle messages,\ serverVars,\ candidateVars,\ leaderVars,\ logVars,\ witnessVars\rangle$

Helpers

$NonWitnessServer \triangleq Server \setminus \{WitnessServer\}$

The set of all quorums. This just calculates simple majorities, but the only

important property is that every quorum overlaps with every other.

$Quorum \triangleq \{i \in \text{SUBSET } (Server) : Cardinality(i) * 2 > Cardinality(Server)\}$

The set of all replication sets for each server.

$ReplicationSets \triangleq$
$[i \in NonWitnessServer \mapsto$
$\{s \in \text{SUBSET } (Server) :$
$\land\ i \in s$
$\land\ Cardinality(s) = Cardinality(NonWitnessServer)\}]$

The term of the last entry in a *log*, or 0 if the *log* is empty.

$LastTerm(xlog) \triangleq \text{IF } Len(xlog) = 0 \text{ THEN } 0 \text{ ELSE } xlog[Len(xlog)].term$

The *subterm* of the last entry in a *log*, or 0 if the *log* is empty.

$LastSubterm(xlog) \triangleq \text{IF } Len(xlog) = 0 \text{ THEN } 0 \text{ ELSE } xlog[Len(xlog)].subterm$

Helper for *Send* and *Reply*. Given a message m and bag of messages, return a new bag of messages with one more m in it.

$WithMessage(m, msgs) \triangleq msgs \oplus SetToBag(\{m\})$

Helper for *Discard* and *Reply*. Given a message m and bag of messages, return a new bag of messages with one less m in it.

$WithoutMessage(m, msgs) \triangleq msgs \ominus SetToBag(\{m\})$

Add a message to the bag of messages.

$SendDirect(m) \triangleq messages' = WithMessage(m, messages)$

Remove a message from the bag of messages. Used when a server is done processing a message.

$DiscardDirect(m) \triangleq messages' = WithoutMessage(m, messages)$

Combination of *Send* and *Discard*

$ReplyDirect(response, request) \triangleq$
$\quad messages' = WithoutMessage(request, WithMessage(response, messages))$

$Send(m) \triangleq SendDirect(m)$
$Reply(response, request) \triangleq ReplyDirect(response, request)$
$Discard(m) \triangleq DiscardDirect(m)$

Return the minimum value from a set, or undefined if the set is empty.

$Min(s) \triangleq \text{CHOOSE } x \in s : \forall\, y \in s : x \leq y$

Return the maximum value from a set, or undefined if the set is empty.

$Max(s) \triangleq \text{CHOOSE } x \in s : \forall\, y \in s : x \geq y$
$MaxOrZero(s) \triangleq \text{IF } s = \{\} \text{ THEN } 0 \text{ ELSE } Max(s)$

Define initial values for all variables

$$\begin{aligned} InitHistoryVars \triangleq\ & \land\ elections = \{\} \\ & \land\ voterLog = [i \in Server \mapsto [j \in \{\} \mapsto \langle\rangle]] \\ & \land\ witnessAcks = \{\} \end{aligned}$$
$InitServerVars \triangleq \land\ currentTerm = [i \in Server \mapsto 1]$

$$
\begin{aligned}
&\land\ state &&= [i \in Server \mapsto Follower] \\
&\land\ votedFor &&= [i \in Server \mapsto Nil] \\
InitCandidateVars \triangleq\ &\land\ votesResponded &&= [i \in Server \mapsto \{\}] \\
&\land\ votesGranted &&= [i \in Server \mapsto \{\}]
\end{aligned}
$$

The values $nextIndex[i][i]$ and $matchIndex[i][i]$ are never read, since the leader does not send itself messages. It's still easier to include these in the functions.

$$
\begin{aligned}
InitLeaderVars \triangleq\ &\land\ nextIndex &&= [i \in Server \mapsto [j \in Server \mapsto 1]] \\
&\land\ matchIndex &&= [i \in Server \mapsto [j \in Server \mapsto 0]] \\
&\land\ replicationSet &&= [i \in Server \mapsto NonWitnessServer] \\
&\land\ currentSubterm &&= [i \in Server \mapsto 0] \\
&\land\ witnessSubterm &&= [i \in Server \mapsto 0]
\end{aligned}
$$

$$
\begin{aligned}
InitLogVars \triangleq\ &\land\ log &&= [i \in Server \mapsto \langle\rangle] \\
&\land\ commitIndex &&= [i \in Server \mapsto 0]
\end{aligned}
$$

$$
\begin{aligned}
InitWitnessVars \triangleq\ &\land\ witnessReplicationSet = NonWitnessServer \\
&\land\ witnessLastLogTerm = 0 \\
&\land\ witnessLastLogSubterm = 0
\end{aligned}
$$

$$
\begin{aligned}
Init \triangleq\ &\land\ messages = EmptyBag \\
&\land\ InitHistoryVars \\
&\land\ InitServerVars \\
&\land\ InitCandidateVars \\
&\land\ InitLeaderVars \\
&\land\ InitLogVars \\
&\land\ InitWitnessVars
\end{aligned}
$$

Define state transitions

Server i restarts from stable storage.

It loses everything but its *currentTerm*, *votedFor*, and *log*.

$$
\begin{aligned}
Restart(i) \triangleq\ &\land\ i &&\neq WitnessServer \\
&\land\ state' &&= [state \text{ EXCEPT } ![i] = Follower] \\
&\land\ votesResponded' &&= [votesResponded \text{ EXCEPT } ![i] = \{\}] \\
&\land\ votesGranted' &&= [votesGranted \text{ EXCEPT } ![i] = \{\}] \\
&\land\ voterLog' &&= [voterLog \text{ EXCEPT } ![i] = [j \in \{\} \mapsto \langle\rangle]] \\
&\land\ nextIndex' &&= [nextIndex \text{ EXCEPT } ![i] = [j \in Server \mapsto 1]]
\end{aligned}
$$

$\land$ $matchIndex'$ $= [matchIndex$ EXCEPT $![i] = [j \in Server \mapsto 0]]$
$\land$ $currentSubterm'$ $= [currentSubterm$ EXCEPT $![i] = 0]$
$\land$ $replicationSet'$ $= [replicationSet$ EXCEPT $![i] = NonWitnessServer]$
$\land$ $witnessSubterm'$ $= [witnessSubterm$ EXCEPT $![i] = 0]$
$\land$ UNCHANGED $\langle messages, currentTerm, commitIndex, witnessAcks, votedFor, log, elections, witnessVar$

Server i times out and starts a new election.

$Timeout(i) \triangleq$ $\land i \neq WitnessServer$
$\land state[i] \in \{Follower, Candidate\}$
$\land state' = [state$ EXCEPT $![i] = Candidate]$
$\land currentTerm' = [currentTerm$ EXCEPT $![i] = currentTerm[i] + 1]$
$\land votedFor' = [votedFor$ EXCEPT $![i] = Nil]$
$\land votesResponded' = [votesResponded$ EXCEPT $![i] = \{\}]$
$\land votesGranted' = [votesGranted$ EXCEPT $![i] = \{\}]$
$\land voterLog' = [voterLog$ EXCEPT $![i] = [j \in \{\} \mapsto \langle\rangle]]$
$\land$ UNCHANGED $\langle messages, leaderVars, logVars, witnessVars\rangle$

Candidate i sends j a *RequestVote* request.

$RequestVote(i, j) \triangleq$
$\land state[i] = Candidate$
$\land j \neq WitnessServer$
$\land j \notin votesResponded[i]$
$\land Send([mtype \mapsto RequestVoteRequest,$
$mterm \mapsto currentTerm[i],$
$mlastLogTerm \mapsto LastTerm(log[i]),$
$mlastLogIndex \mapsto Len(log[i]),$
$msource \mapsto i,$
$mdest \mapsto j])$
$\land$ UNCHANGED $\langle serverVars, candidateVars, leaderVars, logVars, witnessVars\rangle$

Candidate i sends witness a *RequestWitnessVote* request.

$RequestWitnessVote(i) \triangleq$
$\land state[i] = Candidate$
$\land WitnessServer \notin votesResponded[i]$
$\land (votesGranted[i] \cup \{WitnessServer\}) \in Quorum$
$\land Send([mtype \mapsto RequestWitnessVoteRequest,$
$mterm \mapsto currentTerm[i],$
$mlastLogTerm \mapsto LastTerm(log[i]),$

```
                    mlastLogSubterm   ↦ LastSubterm(log[i]),
                    mvotesGranted     ↦ votesGranted[i],
                    msource           ↦ i,
                    mdest             ↦ WitnessServer])
    ∧ UNCHANGED ⟨serverVars, candidateVars, leaderVars, logVars,
                 witnessVars⟩
```

Leader i sends j an *AppendEntries* request containing up to 1 entry.
While implementations may want to send more than 1 at a time, this spec uses
just 1 because it minimizes atomic regions without loss of generality.

```
AppendEntries(i, j) ≜
    ∧ i ≠ j
    ∧ state[i] = Leader
    ∧ LET prevLogIndex ≜ nextIndex[i][j] − 1
          prevLogTerm ≜ IF prevLogIndex > 0 THEN
                           log[i][prevLogIndex].term
                       ELSE
                           0
```

Send up to 1 entry, constrained by the end of the *log*.

```
          lastEntry ≜ Min({Len(log[i]), nextIndex[i][j]})
          entries ≜ SubSeq(log[i], nextIndex[i][j], lastEntry)
      IN  Send([mtype           ↦ AppendEntriesRequest,
                mterm           ↦ currentTerm[i],
                mprevLogIndex   ↦ prevLogIndex,
                mprevLogTerm    ↦ prevLogTerm,
                mentries        ↦ entries,
```

mlog is used as a history variable for the proof.
It would not exist in a real implementation.

```
                mlog            ↦ log[i],
                mcommitIndex    ↦ Min({commitIndex[i], lastEntry}),
                msource         ↦ i,
                mdest           ↦ j])
    ∧ UNCHANGED ⟨serverVars, candidateVars, leaderVars, logVars,
                 witnessVars⟩
```

Leader i sends witness an *AppendEntriesToWitness* request with *log* entry
in current *subterm* acknowledged by a subquorum in current replication set.

$AppendEntriesToWitness(i) \triangleq$

$\land\ state[i] = Leader$

$\land\ WitnessServer \in replicationSet[i]$

$\land$ LET $Agree(index) \triangleq$

$\{i\} \cup \{k \in replicationSet[i] : matchIndex[i][k] \geq index\}$

$IsAgreed(k) \triangleq$

$\land\ log[i][k].term = currentTerm[i]$

$\land\ log[i][k].subterm = currentSubterm[i]$

$\land\ (\{WitnessServer\} \cup Agree(k)) \in Quorum$

$agreeIndexes \triangleq \{k \in 1\,..\,Len(log[i]) : IsAgreed(k)\}$

$lastEntry \triangleq Max(agreeIndexes)$

IN $\land\ agreeIndexes \neq \{\}$

$\land\ \lor$ *witnessSubterm* never exceeds *currentSubterm* following specification

$\land\ currentSubterm[i] > witnessSubterm[i]$

$\land\ Send([mtype \mapsto AppendEntriesToWitnessRequest,$

$mterm \mapsto currentTerm[i],$

$mlogTerm \mapsto log[i][lastEntry].term,$

$mlogSubterm \mapsto log[i][lastEntry].subterm,$

$mreplicationSet \mapsto replicationSet[i],$

$mindex \mapsto lastEntry,$

mlog and *mentries* are used as history variable for the proof. They do not exist in a real implementation.

$mlog \mapsto log[i],$

$mentries \mapsto SubSeq(log[i], 1, lastEntry),$

$msource \mapsto i,$

$mdest \mapsto WitnessServer])$

$\lor$ shortcut replication

$\land\ currentSubterm[i] = witnessSubterm[i]$

$\land\ Send([mtype \mapsto AppendEntriesResponse,$

$mterm \mapsto currentTerm[i],$

$msuccess \mapsto$ TRUE,

$mmatchIndex \mapsto lastEntry,$

$msource \mapsto WitnessServer,$

$mdest \mapsto i,$

$mshortcut \mapsto$ TRUE$])$

$\land$ UNCHANGED $\langle serverVars, candidateVars, leaderVars, logVars, witnessVars\rangle$

Candidate i transitions to leader.

$$
\begin{aligned}
&BecomeLeader(i) \triangleq \\
&\quad \land state[i] = Candidate \\
&\quad \land votesGranted[i] \in Quorum \\
&\quad \land state' = [state \text{ EXCEPT } ![i] = Leader] \\
&\quad \land nextIndex' = [nextIndex \text{ EXCEPT } ![i] = \\
&\qquad\qquad [j \in Server \mapsto Len(log[i]) + 1]] \\
&\quad \land matchIndex' = [matchIndex \text{ EXCEPT } ![i] = \\
&\qquad\qquad [j \in Server \mapsto 0]] \\
&\quad \land currentSubterm' = [currentSubterm \text{ EXCEPT } ![i] = 0] \\
&\quad \land replicationSet' = [replicationSet \text{ EXCEPT } ![i] = NonWitnessServer] \\
&\quad \land witnessSubterm' = [witnessSubterm \text{ EXCEPT } ![i] = 0] \\
&\quad \land elections' = elections \cup \\
&\qquad\qquad \{[eterm \mapsto currentTerm[i], \\
&\qquad\qquad\ \ eleader \mapsto i, \\
&\qquad\qquad\ \ elog \mapsto log[i], \\
&\qquad\qquad\ \ evotes \mapsto votesGranted[i], \\
&\qquad\qquad\ \ evoterLog \mapsto voterLog[i]]\} \\
&\quad \land \text{UNCHANGED } \langle messages, currentTerm, witnessAcks, votedFor, candidateVars, logVars, witnessVars \rangle
\end{aligned}
$$

Leader i receives a client request to add v to the *log*.

$$
\begin{aligned}
&ClientRequest(i, v) \triangleq \\
&\quad \land state[i] = Leader \\
&\quad \land \text{LET } entry \triangleq [term \mapsto currentTerm[i], \\
&\qquad\qquad\qquad\qquad\ \ subterm \mapsto currentSubterm[i], \\
&\qquad\qquad\qquad\qquad\ \ value \mapsto v] \\
&\qquad\quad newLog \triangleq Append(log[i], entry) \\
&\qquad \text{IN } log' = [log \text{ EXCEPT } ![i] = newLog] \\
&\quad \land \text{UNCHANGED } \langle messages, serverVars, candidateVars, \\
&\qquad\qquad\qquad leaderVars, commitIndex, witnessVars \rangle
\end{aligned}
$$

Leader i advances its *commitIndex*.
This is done as a separate step from handling *AppendEntries* responses,
in part to minimize atomic regions, and in part so that leaders of
single-server clusters are able to mark entries committed.

$$
\begin{aligned}
&AdvanceCommitIndex(i) \triangleq \\
&\quad \land state[i] = Leader
\end{aligned}
$$

```
    ∧ LET  The set of servers that agree up through index.
           Agree(index) ≜ {i} ∪ {k ∈ Server :
                                  matchIndex[i][k] ≥ index}
           The maximum indexes for which a quorum agrees
           agreeIndexes ≜ {index ∈ 1 .. Len(log[i]) :
                              Agree(index) ∈ Quorum}
           New value for commitIndex'[i]
           newCommitIndex ≜
              IF ∧ agreeIndexes ≠ {}
                 ∧ log[i][Max(agreeIndexes)].term = currentTerm[i]
                 ∧ log[i][Max(agreeIndexes)].subterm = currentSubterm[i]
              THEN
                 Max(agreeIndexes)
              ELSE
                 commitIndex[i]
       IN  commitIndex' = [commitIndex EXCEPT ![i] = newCommitIndex]
    ∧ UNCHANGED ⟨messages, serverVars, candidateVars, leaderVars, log, witnessVars⟩

 Adjust replication set on leader i. This action updates leader's replication
 set by a new one.
AdjustReplicationSet(i, s) ≜
    ∧ state[i] = Leader
    ∧ replicationSet[i] ≠ s
    ∧ replicationSet'    = [replicationSet EXCEPT ![i] = s]
    ∧ currentSubterm'    = [currentSubterm EXCEPT ![i] = @ + 1]
    ∧ UNCHANGED ⟨messages, serverVars, candidateVars, nextIndex, matchIndex,
                 elections, witnessSubterm, witnessAcks, log, commitIndex, witnessVars⟩

 Message handlers
 i = recipient, j = sender, m = message

 Server i receives a RequestVote request from server j with
 m.mterm ≤ currentTerm[i].
HandleRequestVoteRequest(i, j, m) ≜
   LET logOk ≜  ∨ m.mlastLogTerm > LastTerm(log[i])
                ∨ ∧ m.mlastLogTerm = LastTerm(log[i])
                  ∧ m.mlastLogIndex ≥ Len(log[i])
```

```
        grant  ≜  ∧ m.mterm = currentTerm[i]
                  ∧ logOk
                  ∧ votedFor[i] ∈ {Nil, j}
    IN  ∧ m.mterm ≤ currentTerm[i]
        ∧ ∨ grant   ∧ votedFor' = [votedFor EXCEPT ![i] = j]
          ∨ ¬grant ∧ UNCHANGED votedFor
        ∧ Reply([mtype          ↦ RequestVoteResponse,
                 mterm          ↦ currentTerm[i],
                 mvoteGranted   ↦ grant,
                  mlog is used just for the elections history variable for
                  the proof. It would not exist in a real implementation.
                 mlog           ↦ log[i],
                 msource        ↦ i,
                 mdest          ↦ j],
                 m)
        ∧ UNCHANGED ⟨state, currentTerm, candidateVars, leaderVars, logVars, witnessVars⟩

 Witness receives a RequestWitnessVote request from server j with
 m.mterm ≤ currentTerm[WitnessServer].
HandleRequestWitnessVoteRequest(j, m) ≜
    LET logOk  ≜  ∨ m.mlastLogTerm > witnessLastLogTerm
                  ∨ ∧ m.mlastLogTerm = witnessLastLogTerm
                    ∧ m.mlastLogSubterm > witnessLastLogSubterm
                  ∨ ∧ m.mlastLogTerm = witnessLastLogTerm
                    ∧ m.mlastLogSubterm = witnessLastLogSubterm
                    ∧ m.mvotesGranted ⊆ witnessReplicationSet
        grant  ≜  ∧ m.mterm = currentTerm[WitnessServer]
                  ∧ logOk
                  ∧ votedFor[WitnessServer] ∈ {Nil, j}
    IN  ∧ m.mterm ≤ currentTerm[WitnessServer]
        ∧ ∨ grant   ∧ votedFor' = [votedFor EXCEPT ![WitnessServer] = j]
          ∨ ¬grant ∧ UNCHANGED votedFor
        ∧ Reply([mtype          ↦ RequestVoteResponse,
                 mterm          ↦ currentTerm[WitnessServer],
                 mvoteGranted   ↦ grant,
                  mlog is used just for the elections history variable for
                  the proof. It would not exist in a real implementation.
```

$mlog \mapsto log[WitnessServer],$
$msource \mapsto WitnessServer,$
$mdest \mapsto j],$
$m)$
$\land$ UNCHANGED $\langle state,\ currentTerm,\ candidateVars,\ leaderVars,\ logVars,\ witnessVars\rangle$

Server i receives a *RequestVote* response from server j with $m.mterm = currentTerm[i]$.

$HandleRequestVoteResponse(i,\ j,\ m) \triangleq$
This tallies votes even when the current state is not *Candidate*, but they won't be looked at, so it doesn't matter.
$\land\ m.mterm = currentTerm[i]$
$\land\ votesResponded' = [votesResponded$ EXCEPT $![i] = votesResponded[i] \cup \{j\}]$
$\land\ \lor\ \land\ m.mvoteGranted$
$\land\ votesGranted' = [votesGranted$ EXCEPT $![i] = votesGranted[i] \cup \{j\}]$
$\land\ voterLog' = [voterLog$ EXCEPT $![i] = voterLog[i]$ @@ $(j :> m.mlog)]$
$\lor\ \land\ \neg m.mvoteGranted$
$\land$ UNCHANGED $\langle votesGranted,\ voterLog\rangle$
$\land\ Discard(m)$
$\land$ UNCHANGED $\langle serverVars,\ votedFor,\ leaderVars,\ logVars,\ witnessVars\rangle$

Server i receives an *AppendEntries* request from server j with $m.mterm \leq currentTerm[i]$. This just handles $m.entries$ of length 0 or 1, but implementations could safely accept more by treating them the same as multiple independent requests of 1 entry.

$HandleAppendEntriesRequest(i,\ j,\ m) \triangleq$
LET $logOk \triangleq \lor\ m.mprevLogIndex = 0$
$\lor\ \land\ m.mprevLogIndex > 0$
$\land\ m.mprevLogIndex \leq Len(log[i])$
$\land\ m.mprevLogTerm = log[i][m.mprevLogIndex].term$
IN $\land\ m.mterm \leq currentTerm[i]$
$\land\ \lor\ \land$ reject request
$\lor\ m.mterm < currentTerm[i]$

```
        ∨ ∧ m.mterm = currentTerm[i]
          ∧ state[i] = Follower
          ∧ ¬logOk
      ∧ Reply([mtype          ↦ AppendEntriesResponse,
               mterm          ↦ currentTerm[i],
               msuccess       ↦ FALSE,
               mmatchIndex    ↦ 0,
               msource        ↦ i,
               mdest          ↦ j],
               m)
      ∧ UNCHANGED ⟨serverVars, logVars⟩
    ∨ return to follower state
      ∧ m.mterm = currentTerm[i]
      ∧ state[i] = Candidate
      ∧ state' = [state EXCEPT ![i] = Follower]
      ∧ UNCHANGED ⟨currentTerm, votedFor, logVars, messages⟩
    ∨ accept request
      ∧ m.mterm = currentTerm[i]
      ∧ state[i] = Follower
      ∧ logOk
      ∧ LET index ≜ m.mprevLogIndex + 1
        IN  ∨ already done with request
              ∧ ∨ m.mentries = ⟨⟩
                ∨ ∧ m.mentries ≠ ⟨⟩
                  ∧ Len(log[i]) ≥ index
                  ∧ log[i][index].term = m.mentries[1].term
                This could make our commitIndex decrease (for
                example if we process an old, duplicated request),
                but that doesn't really affect anything.
              ∧ commitIndex' = [commitIndex EXCEPT ![i] =
                                m.mcommitIndex]
              ∧ Reply([mtype          ↦ AppendEntriesResponse,
                       mterm          ↦ currentTerm[i],
                       msuccess       ↦ TRUE,
                       mmatchIndex    ↦ m.mprevLogIndex +
                                        Len(m.mentries),
                       msource        ↦ i,
```

```
                                mdest      ↦ j],
                                m)
                        ∧ UNCHANGED ⟨serverVars, log⟩
                    ∨ conflict: remove 1 entry
                        ∧ m.mentries ≠ ⟨⟩
                        ∧ Len(log[i]) ≥ index
                        ∧ log[i][index].term  ≠ m.mentries[1].term
                        ∧ LET new ≜ [index2 ∈ 1 .. (Len(log[i]) − 1) ↦
                                        log[i][index2]]
                          IN  log′ = [log EXCEPT ![i] = new]
                        ∧ UNCHANGED ⟨serverVars, commitIndex, messages⟩
                    ∨ no conflict: append entry
                        ∧ m.mentries ≠ ⟨⟩
                        ∧ Len(log[i]) = m.mprevLogIndex
                        ∧ log′ = [log EXCEPT ![i] =
                                    Append(log[i], m.mentries[1])]
                        ∧ UNCHANGED ⟨serverVars, commitIndex, messages⟩
                ∧ UNCHANGED ⟨candidateVars, leaderVars, witnessVars⟩

Witness receives an AppendEntriesToWitnessRequest from server j with
m.mterm ≤ currentTerm[WitnessServer].
HandleAppendEntriesToWitnessRequest(j, m) ≜
    ∧ m.mterm ≤ currentTerm[WitnessServer]
    ∧ ∨ reject request
        ∧ m.mterm < currentTerm[WitnessServer]
        ∧ Reply([mtype      ↦ AppendEntriesResponse,
                 mterm      ↦ currentTerm[WitnessServer],
                 msuccess   ↦ FALSE,
                 mmatchIndex ↦ 0,
                 msource    ↦ WitnessServer,
                 mdest      ↦ j],
                 m)
        ∧ UNCHANGED ⟨serverVars, logVars, witnessVars⟩
      ∨ accept request, always no conflict.
        ∧ m.mterm = currentTerm[WitnessServer]
        ∧ ∨ m.mlogTerm > witnessLastLogTerm
          ∨ ∧ m.mlogTerm = witnessLastLogTerm
```

$\land\ m.mlogSubterm > witnessLastLogSubterm$

$\land\ witnessReplicationSet' = m.mreplicationSet$

$\land\ witnessLastLogTerm' = m.mlogTerm$

$\land\ witnessLastLogSubterm' = m.mlogSubterm$

log will not be modified in real implementation. It is only used here for the proof.

$\land\ log' = [log \text{ EXCEPT } ![WitnessServer] = m.mentries]$

$\land\ Reply([mtype \mapsto AppendEntriesResponse,$
$mterm \mapsto currentTerm[WitnessServer],$
$msuccess \mapsto \text{TRUE},$
$mmatchIndex \mapsto m.mindex,$
$msource \mapsto WitnessServer,$
$mdest \mapsto j],$
$m)$

$\land\ \text{UNCHANGED } \langle serverVars,\ commitIndex\rangle$

$\land\ \text{UNCHANGED } \langle candidateVars,\ leaderVars\rangle$

Server i receives an *AppendEntries* response from server j with $m.mterm = currentTerm[i]$.

$HandleAppendEntriesResponse(i,\ j,\ m) \triangleq$

$\land\ m.mterm = currentTerm[i]$

$\land\ \lor\ \land\ m.msuccess$ successful

$\land\ nextIndex' = [nextIndex \text{ EXCEPT } ![i][j] = Max(\{@,\ m.mmatchIndex + 1\})]$

$\land\ matchIndex' = [matchIndex \text{ EXCEPT } ![i][j] = Max(\{@,\ m.mmatchIndex\})]$

$\land\ \text{IF } j = WitnessServer \land log[i][m.mmatchIndex].subterm = currentSubterm[i] \text{ THEN}$
$witnessSubterm' = [witnessSubterm \text{ EXCEPT } ![i] = currentSubterm[i]]$
ELSE
UNCHANGED $\langle witnessSubterm\rangle$

$\land\ \text{IF } j = WitnessServer \land \text{“mshortcut”} \notin \text{DOMAIN } m \text{ THEN}$
$witnessAcks' = witnessAcks \cup \{\langle m.mterm,\ m.mmatchIndex\rangle\}$
ELSE
UNCHANGED $\langle witnessAcks\rangle$

$\lor\ \land\ \neg m.msuccess$ not successful

$\land\ nextIndex' = [nextIndex \text{ EXCEPT } ![i][j] =$
$Max(\{nextIndex[i][j] - 1,\ 1\})]$

$\land\ \text{UNCHANGED } \langle matchIndex,\ witnessSubterm,\ witnessAcks\rangle$

$\land\ Discard(m)$

```
    ∧ UNCHANGED ⟨serverVars, candidateVars, logVars, elections,
                 currentSubterm, replicationSet, witnessVars⟩

 Any RPC with a newer term causes the recipient to advance its term first.
UpdateTerm(i, j, m) ≜
    ∧ m.mterm > currentTerm[i]
    ∧ currentTerm'    = [currentTerm EXCEPT ![i] = m.mterm]
    ∧ state'          = [state       EXCEPT ![i] = Follower]
    ∧ votedFor'       = [votedFor    EXCEPT ![i] = Nil]
       messages is unchanged so m can be processed further.
    ∧ UNCHANGED ⟨messages, candidateVars, leaderVars, logVars, witnessVars⟩

 Responses with stale terms are ignored.
DropStaleResponse(i, j, m) ≜
    ∧ m.mterm < currentTerm[i]
    ∧ Discard(m)
    ∧ UNCHANGED ⟨serverVars, candidateVars, leaderVars, logVars, witnessVars⟩

 Receive a message.
Receive(m) ≜
    LET i ≜ m.mdest
        j ≜ m.msource
    IN   Any RPC with a newer term causes the recipient to advance
         its term first. Responses with stale terms are ignored.
        ∨ UpdateTerm(i, j, m)
        ∨ ∧ m.mtype = RequestVoteRequest
          ∧ HandleRequestVoteRequest(i, j, m)
        ∨ ∧ m.mtype = RequestWitnessVoteRequest
          ∧ HandleRequestWitnessVoteRequest(j, m)
        ∨ ∧ m.mtype = RequestVoteResponse
          ∧ ∨ DropStaleResponse(i, j, m)
            ∨ HandleRequestVoteResponse(i, j, m)
        ∨ ∧ m.mtype = AppendEntriesRequest
          ∧ HandleAppendEntriesRequest(i, j, m)
        ∨ ∧ m.mtype = AppendEntriesToWitnessRequest
          ∧ HandleAppendEntriesToWitnessRequest(j, m)
        ∨ ∧ m.mtype = AppendEntriesResponse
          ∧ ∨ DropStaleResponse(i, j, m)
```

$\lor$ *HandleAppendEntriesResponse*$(i,\ j,\ m)$

End of message handlers.

Network state transitions

The network duplicates a message

DuplicateMessage(m) $\triangleq$

$\land$ *Send*(m)

$\land$ UNCHANGED $\langle$*serverVars*, *candidateVars*, *leaderVars*, *logVars*, *witnessVars*$\rangle$

The network drops a message

DropMessage(m) $\triangleq$

$\land$ *Discard*(m)

$\land$ UNCHANGED $\langle$*serverVars*, *candidateVars*, *leaderVars*, *logVars*, *witnessVars*$\rangle$

History variable that tracks every *log* ever:

allLogs $\triangleq$ $\{log[i] : i \in Server\}$

NextRequestVoteRequest $\triangleq$ $\exists\, m \in$ DOMAIN *messages* : $m.mtype = RequestVoteRequest \land Receive(m)$

NextRequestVoteResponse $\triangleq$ $\exists\, m \in$ DOMAIN *messages* : $m.mtype = RequestVoteResponse \land Receive(m)$

NextRequestWitnessVoteRequest $\triangleq$ $\exists\, m \in$ DOMAIN *messages* : $m.mtype = RequestWitnessVoteRequest \land Rece$

NextAppendEntriesRequest $\triangleq$ $\exists\, m \in$ DOMAIN *messages* : $m.mtype = AppendEntriesRequest \land Receive(m)$

NextAppendEntriesResponse $\triangleq$ $\exists\, m \in$ DOMAIN *messages* : $m.mtype = AppendEntriesResponse \land Receive(m)$

NextAppendEntriesToWitnessRequest $\triangleq$ $\exists\, m \in$ DOMAIN *messages* : $m.mtype = AppendEntriesToWitnessReq$

NextDuplicateMessage $\triangleq$ $\exists\, m \in$ DOMAIN *messages* : *DuplicateMessage*(m)

NextDropMessage $\triangleq$ $\exists\, m \in$ DOMAIN *messages* : *DropMessage*(m)

Defines how the variables may transition.

Next $\triangleq$ $\lor \exists\, i \in NonWitnessServer : Restart(i)$

$\lor \exists\, i \in NonWitnessServer : Timeout(i)$

$\lor \exists\, i, j \in NonWitnessServer : RequestVote(i, j)$

$\lor \exists\, i \in NonWitnessServer : BecomeLeader(i)$

$\lor \exists\, i \in NonWitnessServer,\ v \in Value : ClientRequest(i, v)$

$\lor \exists\, i \in NonWitnessServer : AdvanceCommitIndex(i)$

$\lor \exists\, i, j \in NonWitnessServer : AppendEntries(i, j)$

$\lor \exists\, i \in NonWitnessServer : \exists\, s \in ReplicationSets[i] : AdjustReplicationSet(i, s)$

$\lor \exists\, i \in \mathit{NonWitnessServer} : \mathit{RequestWitnessVote}(i)$
$\lor \exists\, i \in \mathit{NonWitnessServer} : \mathit{AppendEntriesToWitness}(i)$
$\lor \mathit{NextRequestVoteRequest}$
$\lor \mathit{NextRequestVoteResponse}$
$\lor \mathit{NextRequestWitnessVoteRequest}$
$\lor \mathit{NextAppendEntriesRequest}$
$\lor \mathit{NextAppendEntriesResponse}$
$\lor \mathit{NextAppendEntriesToWitnessRequest}$
$\lor \mathit{NextDuplicateMessage}$
$\lor \mathit{NextDropMessage}$

The specification must start with the initial state and transition according to *Next*.

$\mathit{Spec} \triangleq$
$\land \mathit{Init}$
$\land \Box[\mathit{Next}]_{\mathit{vars}}$

The main safety properties are below

Correctness invariants

The prefix of the *log* of server i that has been committed

$\mathit{Committed}(i) \triangleq \text{IF } i = \mathit{WitnessServer} \text{ THEN } \mathit{log}[i] \text{ ELSE } \mathit{SubSeq}(\mathit{log}[i], 1, \mathit{commitIndex}[i])$

The prefix of the *log* of server i that has been committed up to term x

$\mathit{CommittedTermPrefix}(i, x) \triangleq$
Only if *log* of i is non-empty, and if there exists an entry up to the term x
$\text{IF } \mathit{Len}(\mathit{log}[i]) \neq 0 \land \exists\, y \in \text{DOMAIN } \mathit{log}[i] : \mathit{log}[i][y].\mathit{term} \leq x$
THEN
then, we use the subsequence up to the maximum committed term of the leader
$\text{LET } \mathit{maxTermIndex} \triangleq$
$\text{CHOOSE } y \in \text{DOMAIN } \mathit{log}[i] :$
$\land \mathit{log}[i][y].\mathit{term} \leq x$
$\land \forall\, z \in \text{DOMAIN } \mathit{log}[i] : \mathit{log}[i][z].\mathit{term} \leq x \implies y \geq z$
$\text{IN } \mathit{SubSeq}(\mathit{log}[i], 1, \mathit{Min}(\{\mathit{maxTermIndex}, \mathit{commitIndex}[i]\}))$
Otherwise the prefix is the empty tuple
ELSE $\langle\rangle$

```
 Committed log entries should never conflict between servers
LogInv ≜
    ∀ i, j ∈ Server :
        ∨ IsPrefix(Committed(i), Committed(j))
        ∨ IsPrefix(Committed(j), Committed(i))

 Note that LogInv checks for safety violations across space
 This is a key safety invariant and should always be checked
THEOREM Spec ⟹ □LogInv

 There is only ever one leader per term. However, this does not preclude multiple
 servers being leader at the same time, i.e., |{s ∈ Server : state[s] = Leader}| > 1,
 as long as each server is leader in a different term.

 However, this invariant is not violated if two servers *atomically* trade places as
 leader in the same term, i.e., state' = [state EXCEPT ![s] = Follower, ![t] = Leader]
 with state[s] = Leader ∧ state[t] ≠ Leader. For that, we would need a suitable action
 property.
MoreThanOneLeaderInv ≜
    ∀ i, j ∈ Server :
      ( ∧ currentTerm[i] = currentTerm[j]
        ∧ state[i] = Leader
        ∧ state[j] = Leader)
        ⟹ i = j

 If a candidate has a chance of being elected, there
 are no log entries with that candidate's term
CandidateTermNotInLogInv ≜
    ∀ i ∈ Server :
      ( ∧ state[i] = Candidate
        ∧ {j ∈ Server :
              ∧ currentTerm[j] = currentTerm[i]
              ∧ votedFor[j] = i
          } ∈ Quorum
      )
        ⟹
      ∀ j ∈ Server :
      ∀ n ∈ DOMAIN log[j] :
```

$log[j][n].term \neq currentTerm[i]$

A leader always has the greatest index for its current term (this does not
mean all of its *log* will survive if it is not committed)

$ElectionSafetyInv \triangleq$
$\forall\, i \in Server :$
$state[i] = Leader \implies$
$\forall\, j \in Server : i \neq j \implies$
LET $FilterAndMax(a,\, b) \triangleq$
IF $a.term = currentTerm[i]$ THEN $Max(\{a.term,\, b\})$ ELSE b
IN $FoldSeq(FilterAndMax,\, 0,\, log[i]) \geq FoldSeq(FilterAndMax,\, 0,\, log[j])$

Every (index, term) pair determines a *log* prefix.

From page 8 of the Raft paper: "If two logs contain an entry with the same index and term, then the logs are identical in all pr

$LogMatchingInv \triangleq$
$\forall\, i,\, j \in Server : i \neq j \implies$
$\forall\, n \in 1 \,..\, Min(\{Len(log[i]),\, Len(log[j])\}) :$
$log[i][n].term = log[j][n].term \implies$
$SubSeq(log[i],\, 1,\, n) = SubSeq(log[j],\, 1,\, n)$

A leader has all committed entries in its *log*. This is expressed
by *LeaderCompleteness* below. The inductive invariant for
that property is the conjunction of *LeaderCompleteness* with the
other three properties below.

All committed entries are contained in the *log*
of at least one server in every quorum if the last committed entry
is not shortcut replicated.

$QuorumLogInv \triangleq$
$\forall\, i \in Server :$
LET $committed \triangleq Committed(i)$
IN $(\vee\, LastSubterm(committed) = 0$
$\vee\, \langle LastTerm(committed),\, Len(committed)\rangle \in witnessAcks) \implies$
$\forall\, S \in Quorum :$
$\exists\, j \in S :$
$IsPrefix(Committed(i),\, log[j])$

The committed entries in every *log* are a prefix of the
leader's *log* up to the leader's term (since a next *Leader* may already be
elected without the old leader stepping down yet)

$$
\begin{array}{l}
LeaderCompletenessInv \triangleq \\
\quad \forall\, i \in Server : \\
\qquad state[i] = Leader \implies \\
\qquad \forall\, j \in Server : i \neq j \implies \\
\qquad\quad IsPrefix(CommittedTermPrefix(j,\, currentTerm[i]),\, log[i])
\end{array}
$$

Each server's term should be equal to or greater than the terms of messages it has sent

$$
\begin{array}{l}
MonoTermInv \triangleq \\
\quad \forall\, m \in \text{DOMAIN } messages : currentTerm[m.msource] \geq m.mterm
\end{array}
$$

Terms in logs increase monotonically.

$$
\begin{array}{l}
MonoLogInv \triangleq \\
\quad \forall\, i \in Server : \\
\qquad log[i] \neq \langle\rangle \implies \\
\qquad\quad \land\, LastTerm(log[i]) \leq currentTerm[i] \\
\qquad\quad \land\, \forall\, k \in 1 \,..\, Len(log[i]) - 1 : log[i][k].term \leq log[i][k+1].term
\end{array}
$$

Properties

Each server's committed *log* is append-only

$$
\begin{array}{l}
CommittedLogAppendOnlyProp \triangleq \\
\quad \Box[\forall\, i \in Server : \\
\qquad IsPrefix(Committed(i),\, Committed(i)')]_{vars}
\end{array}
$$

Note that *CommittedLogAppendOnlyProp* checks for safety violations across time
This is a key safety invariant and should always be checked

$$\text{THEOREM } Spec \implies CommittedLogAppendOnlyProp$$

Each server's commit index is monotonically increasing
This is weaker form of *CommittedLogAppendOnlyProp* so it is not checked by default

$$
\begin{array}{l}
MonotonicCommitIndexProp \triangleq \\
\quad \Box[\forall\, i \in Server : \\
\qquad commitIndex'[i] \geq commitIndex[i]]_{vars}
\end{array}
$$

$$MonotonicTermProp \triangleq$$

$\Box[\forall\, i \in Server :$
$currentTerm'[i] \geq currentTerm[i]]_{vars}$

$MonotonicMatchIndexProp \triangleq$

Figure 2, page 4 in the raft paper:
"Volatile state on leaders, reinitialized after election. For each server,
index of the highest *log* entry known to be replicated on server. Initialized
to 0, increases monotonically". In other words, *matchIndex* never decrements
unless the current action is a node becoming leader or restarted.

$\Box[(\neg\exists\, i \in NonWitnessServer : \langle BecomeLeader(i) \lor Restart(i)\rangle_{vars}) \implies$
$(\forall\, i \in NonWitnessServer,\, j \in Server :$
$matchIndex'[i][j] \geq matchIndex[i][j])]_{vars}$

$StateTransitionsProp \triangleq$
$\Box[\forall\, i \in Server :$
$\land\, state[i] = Nil \implies state'[i] \in \{Nil,\, Follower\}$
$\land\, state[i] = Follower \implies state'[i] \in \{Follower,\, Candidate\}$
$\land\, state[i] = Candidate \implies state'[i] \in \{Follower,\, Candidate,\, Leader\}$
$\land\, state[i] = Leader \implies state'[i] \in \{Follower,\, Leader\}$
$]_{vars}$

Leader commits *log* entries in during its term and current *subterm*.
$NeverCommitEntryPrevSubtermsProp \triangleq$
$\Box[\forall\, i \in \{s \in Server : state[s] = Leader\} :$
If the *commitIndex* of a leader changes, the *log* entry's term and *subterm* that
the new *commitIndex* points to equals the leader's term and *subterm*, respectively.
$commitIndex'[i] > commitIndex[i] \implies$
$\land\, log[i][commitIndex'[i]].term = currentTerm'[i]$
$\land\, log[i][commitIndex'[i]].subterm = currentSubterm'[i]]_{vars}$

$LogMatchingProp \triangleq$
$\forall\, i,\, j \in Server : \Box\Diamond(log[i] = log[j])$

$LeaderProp \triangleq$
$\Box\Diamond\langle\exists\, i \in Server : state[i] = Leader\rangle_{vars}$

6 Formal proof

In this section, we aim to prove the key properties of the Raft algorithm (namely, Election Safety, Leader Append-Only, Log Matching, Leader Completeness, and State Machine Safety) within the context of the extended Raft algorithm. These properties will be proven to hold true consistently in the extended Raft algorithm. Our proof will build upon the existing proof of the Raft algorithm, and we will reference lemmas from the Raft proof in the format of "Lemma R.n," where 'n' represents the lemma's index in the Raft proof.

Lemma 1. *Following lemmas are true in the extended Raft algorithm.*

1. *Lemma R.1. Each server's currentTerm monotonically increases.*
2. *Lemma R.2. There is at most one leader per term.*
3. *Lemma R.3. A leader's log monotonically grows during its term.*
4. *Lemma R.4. An* $\langle <\rangle index, term >$ *pair identifies a log prefix.*
5. *Lemma R.5. When a follower appends an entry to its log, its log after the append is a prefix of the leader's log at the time the leader sent the AppendEntries request.*
6. *Lemma R.6. A server's current term is always at least as large as the terms in its log.*
7. *Lemma R.7. The terms of entries grow monotonically in each log.*

Proof.

1. Witness, similar to other regular servers, monotonically increases its term upon receiving a message with a larger *mterm* (action *UpdateTerm*). This immediately validates Lemma R.1 as per the specification.
2. In the extended Raft algorithm, leader is elected by a quorum. Neither regular servers nor the witness vote for different servers within the same term, thus maintaining the truth of Lemma R.2 according to its proof.
3. Leader in the extended Raft algorithm manages its log identically to the Raft algorithm, validating Lemma R.3 since
   (a) Actions *BecomeLeader*, *ClientRequest*, and *AppendEntries* behave same as those in the Raft algorithm. Although the log entry is associated with the current subterm, this is trivial to the proof.

(b) *AppendEntriesToWitness* is an implementation of the *AppendEntries* action, with batching log prefix to a log entry (that is associated with the current subterm and has received subquorum acknowledgments from the current replication set).

4. Lemma R.4 and Lemma R.5 are valid in the extended Raft algorithm given that:
   (a) Regular servers replicate logs identically to the Raft algorithm. The existence of a subterm in a log entry is trivial to the proof.
   (b) Leader always sends its log's prefix to the witness.
   (c) Witness replaces its log with the log prefix from the leader, hence $log'[WitnessID]$ remains a prefix of $m.mlog$ for m sent to the witness.
5. The validation of Lemma R.6 in the Raft algorithm relies on the servers' behavior in log replication. Therefore, Lemma R.6 holds true in the extended Raft algorithm as:
   (a) Regular servers replicate logs in an identical manner.
   (b) Witness accepts the request with the necessary condition $currentTerm[WitnessID] = m.mterm$.
   (c) Witness always replaces its log with the log prefix sent from the leader.
6. The proof of Lemma R.7 depends on Lemma R.5 and Lemma R.6. Thus, Lemma R.7 is also valid in the extended Raft algorithm, as both Lemma R.5 and Lemma R.6 are proven true.

□

Now we have Election Safety property (Lemma R.2), Leader Append-Only property (Lemma R.3), and Log Matching property (Lemma R.4) hold in the extended Raft algorithm.
Similarly, we have following lemmas specific for the extended Raft algorithm.

Lemma 2. *Each leader's currentSubterm monotonically increases during each term:*

$$\forall i \in Server :$$
$$currentTerm[i] = currentTerm'[i]$$
$$\implies currentSubterm[i] \leq currentSubterm'[i]$$

Proof. This follows immediately from the specification. □

Lemma 3. *The subterms of entries grow monotonically during its termin each log*

$$\begin{aligned}
&\forall l \in allLogs : \\
&\quad \forall index \in 1..(Len(l) - 1) : \\
&\qquad l[index].term = l[index + 1].term \\
&\qquad\qquad \implies l[index].subterm \leq l[index + 1].subterm
\end{aligned}$$

Proof. 1. Initial state: all logs are empty, so the invariant holds.

2. Inductive step: logs change in one of the following ways:

   (a) Case: a leader adds one entry (client request)

      i. The new entry's subterm is $currentSubterm(leader)$.

      ii. $currentSubterm$ monotonically increases during $currentTerm$.

      iii. Thus, the new entry's subterm is at least as larger as the subterms during $currentTerm$ in this log, since leader's log monotonically grows during its term.

   (b) Case: a follower removes one entry (AppendEntries request)

      i. The invariant still holds, since only the length of the log decreased.

   (c) Case: a follower adds one entry (AppendEntries request), or witness replaces its log by log prefix from leader

      i. $log'[follower]$ is a prefix of $m.mlog$ (by Lemma R.5)

      ii. $m.mlog \in allLogs$

      iii. By the inductive hypothesis, the subterms in $m.mlog$ monotonically grow during its term, so the subterms in $log'[follower]$ monotonically grow during its term.

□

Lemma R.8 is part of the Leader Completeness property in Raft algorithm. Given the existence of shortcut replication, and the fact that witness uses different rules to cast vote, we present proof to the equivalent ***Lemma*** *4 following the same idea as that in the Raft algorithm.*

Lemma 4. *Immediately committed entries are committed*

$$\forall \langle index, term, subterm \rangle \in immediatelyCommitted :$$
$$\langle index, term, subterm \rangle \in committed(term)$$

Proof. 1. Consider an entry $\langle index, term, subterm \rangle$ that is *immediately committed*.

2. Define

$$\begin{aligned} Contradicting \triangleq \{ & election \in elections : \\ & \wedge election.eterm > term \\ & \wedge \langle index, term, subterm \rangle \notin election.elog \} \end{aligned}$$

3. Let *election* be an element in *Contradicting* with a minimal *term* field. That is,

$$\forall\ e \in Contradicting : election.eterm \leq e.eterm.$$

   If more than one election has the same term, choose the earliest one. (The specification does not allow this to happen, but it is safe for a leader to step down and become leader again in the same term)

4. It suffices to show a contradiction, which implies $Contradicting = \phi$.

5. Let $\langle index', term, subterm \rangle$ be any entry that exists in logs of a quorum during *term*, where $index' \leq index$. Such entry must exist if the subterm's replication set contains the witness, since:

   (a) Case $\langle index, term, subterm \rangle$ is immediately committed with no shortcut replication: $\langle index', term, subterm \rangle$ exists in logs of a quorum during *term*, following the specification.

   (b) Case $\langle index, term, subterm \rangle$ is immediately committed with shortcut replication: Following the specification, shortcut replication can only be executed after a log entry in the same subterm being replicated to the witness. Thus $\langle index', term, subterm \rangle$ exists for some $index' < index$.

6. Let *voter* be either

- any regular server that both votes in *election* and contains $\langle index, term, subterm \rangle$ in its log during *term*
- witness that both votes in *election* and contains $\langle index', term, subterm \rangle$ in its log during *term*

Such server must exist, since:

(a) Case: $\langle index, term, subterm \rangle$ is immediately committed with no shortcut replication.

voter must exists, since:

i. $\langle index, term, subterm \rangle$ exists in logs of a quorum during *term*.

ii. A quorum of servers voted in *election* to make it succeed.

iii. Two quorums always overlap.

iv. *voter* can be either regular server or witness.

(b) Case: $\langle index, term, subterm \rangle$ is immediately committed with shortcut replication. *voter* must exists, since:

i. A subquorum of regular servers contains $\langle index, term, subterm \rangle$ in its log during *term*. And the witness server contains $\langle index', term, subterm \rangle$ in its log during *term*. These make a quorum.

ii. A quorum of servers voted in *election* to make it succeed. The server is either regular server or witness.

iii. Two quorums always overlap.

7. Let $voterLog \triangleq election.evoterLog[voter]$, the voter's log at the time it casts its vote.

8. For any entry $\langle k, term, subterm \rangle (k \leq index)$ that the voter contains during *term*, it is also contained by the voter when it casts its vote during $election.eterm$. That is, $\langle k, term, subterm \rangle \in voterLog$, where $k \leq index$:

(a) $\langle k, term, subterm \rangle$ was in the *voter*'s log during *term*.

(b) The *voter* must have stored the entry in *term* before voting in $election.eterm$, since:

i. $election.eterm > term$.

ii. The *voter* rejects requests with terms smaller than its current term, and its current term monotonically increases.

(c) The *voter* couldn't have removed the entry before casting its vote:

i. Case: No *AppendEntriesRequest* with $mterm < term$ removes the entry from the voter's log, since $currentTerm[voter] \geq term$ upon storing the entry, and the voter rejects requests with terms smaller than $currentTerm[voter]$.

ii. Case: No *AppendEntriesRequest* with $mterm = term$ removes the entry from the voter's log, since:

   A. There is only one leader of *term*.

   B. The leader of *term* created and therefore contains the entry.

   C. The leader would not send any conflicting requests to *voter* during *term*.

iii. Case: No *AppendEntriesRequest* with $mterm > term$ removes the entry from the voter's log, since:

   A. Case: $mterm > election.eterm$:
   This can't happen, since $currentTerm[voter] > election.eterm$ would have presented the voter from voting in *term*.

   B. Case: $mterm = election.eterm$:
   Since there is at most one leader per term, this request would have to come from $election.eleader$ as a result of an earlier election in the same term ($election.eterm$).
   Because a leader's log grows monotonically during its term, the leader could not have had $\langle k, term, subterm\rangle$ in its log at the start of its term, hence it could not have had $\langle index, term, subterm\rangle$ because $index \geq k$. Then there exists an earlier election with the same term in *Contradicting*; this is a contradiction.

   C. Case $mterm < election.eterm$:
   The leader of *mterm* must have contained $\langle index, term, subterm\rangle$ (otherwise its election would also be *Contradicting* but have a smaller term than *election*, which is a contradiction). Thus, the leader of *mterm* could not send any conflicting entries to the voter for this index, nor could it send any conflicting entries for prior indexes: that it has this entry implies that it has the entrire prefix before it (Raft lemma 4).

9. Thus, we have $\langle index, term, subterm\rangle \in voterLog$ for non-witness voter, and $\langle index', term, subterm\rangle \in voterLog$ for witness voter.

   We show contradiction in following cases which will then be used to show

contradiction in each case of voting rules.

10. Case: $LastTerm(election.elog) = LastTerm(voterLog) \wedge Len(election.elog) \geq index$

    (a) The leader of $LastTerm(voterLog)$ monotonically grew its log during its term (by Lemma R.3).

    (b) The same leader must have had $election.elog$ as its log at some point, since it created the last entry.

    (c) Thus, $voterLog[1..index]$ is a prefix of $election.elog$.

    (d) Then $\langle index, term, subterm \rangle \in election.elog$, since $\langle index, term, subterm \rangle \in voterLog[1..index]$.

    (e) But $election \in Contradicting$ implies that $\langle index, term, subterm \rangle \notin election.elog$.

11. Case: $LastTerm(election.elog) > term$

    (a) $election.eterm > LastTerm(election.elog)$ since servers increment their $currentTerm$ when starting an election, and Lemma R.6 states that a server's $currentTerm$ is at least as large as the terms in its log.

    (b) Let $prior$ be the election in $elections$ with $prior.eterm = LastTerm(election.elog)$. Such an election must exist since $LastTerm(election.elog) > 0$ and a server must win an election before creating an entry.

    (c) By transitivity, we now have the following inequalities:

$$term \leq LastTerm(election.elog) = prior.eterm < election.eterm$$

    (d) $\langle index, term, subterm \rangle \in prior.elog$, since $prior \notin Contradicting$ ($election$ was assumed to have the lowest term of any election in $Contradicting$, and $prior.eterm < election.eterm$).

    (e) $prior.elog$ is a prefix of $election.elog$ since:

        i. $prior.eleader$ creates entries with $prior.eterm$ by appending them to its log, which monotonicallygrows during $prior.eterm$ from $prior.elog$.

        ii. Thus, any entry with term $prior.eterm$ must follow $prior.elog$ in all logs (by Lemma R.4).

iii. $LastTerm(election.elog) = prior.eterm$

(f) $\langle index, term, subterm \rangle \in election.elog$. Note that this is true no matter $\langle index, term, subterm \rangle$ existing in *voterLog* or not.

(g) This is a contradiction, since *election.elog* was assumed to not contain the committed entry ($election \in Contradicting$).

12. The log comparison during elections states the following, since *voter* granted its vote during *election*:

$$
\begin{array}{l}
\lor\ \land\ voter \neq WitnessID \\
\quad \land\ \lor\ LastTerm(election.elog) > LastTerm(voterLog) \\
\qquad \lor\ \land\ LastTerm(election.elog) = LastTerm(voterLog) \\
\qquad\quad \land\ Len(election.elog) \geq Len(voterLog) \\
\lor\ \land\ voter = WitnessID \\
\quad \land\ \lor\ LastTerm(election.elog) > LastTerm(voterLog) \\
\qquad \lor\ \land\ LastTerm(election.elog) = LastTerm(voterLog) \\
\qquad\quad \land\ LastSubterm(election.elog) > LastSubterm(voterLog) \\
\qquad \lor\ \land\ LastTerm(election.elog) = LastTerm(voterLog) \\
\qquad\quad \land\ LastSubterm(election.elog) = LastSubterm(voterLog) \\
\qquad\quad \land\ election.evoters \setminus WitnessID \subset ReplicationSet
\end{array}
$$

ReplicationSet is replication set of $LastSubterm(voterLog)$ in $LastTerm(voterLog)$ if *voter* is witness.

We now investigate each voting rule in following cases, and use the result of above two cases to show contradiction.

13. Case: *voter* is a regular server, and $LastTerm(election.elog) = LastTerm(voterLog) \land Len(election.elog) \geq Len(voterLog)$

(a) $Len(election.elog) \geq Len(voterLog) \implies Len(election.elog) \geq index$

(b) This leads to contradiction following case 10

14. Case: $LastTerm(election.elog) > LastTerm(voterLog)$, no matter *voter* being a regular server or witness

(a) $LastTerm(election.elog) > LastTerm(voterLog) \implies LastTerm(election.elog) > term$

(b) This leads to contradiction following case 11

15. Case: *voter* is witness, and $LastTerm(election.elog) = LastTerm(voterLog) \wedge LastSubterm(election.elog) > LastSubterm(voterLog)$

    (a) $LastSubterm(election.elog) > LastSubterm(voterLog) \implies Len(election.elog) > Len(voterLog) > term$, following Lemma 3

    (b) This leads to contradiction following case 10

16. Case: *voter* is witness, and

$$\wedge\ LastTerm(election.elog) = LastTerm(voterLog)$$
$$\wedge\ LastSubterm(election.elog) = LastSubterm(voterLog)$$
$$\wedge\ election.evoters \cup WitnessID \subset ReplicationSet$$

    (a) $LastTerm(voterLog) = term \wedge LastSubterm(voterLog) = subterm$

        i. There must be $voter'$ which is a regular server that contains $\langle index, term, subterm \rangle$ and votes in *election*, since

            A. $\langle index, term, subterm \rangle$ exists in at least subquorum regular servers in replication set of *term* and *subterm*, no matter if it is shortcut replicated or not.

            B. A subquorum in replication set of *term* and *subterm* vote in *election*.

            C. Subquorums of regular servers overlap in a replication set.

        ii. This leads to contradiction following case 13 and 14

    (b) $LastTerm(voterLog) = term \wedge LastSubterm(voterLog) > subterm$

        i. $LastTerm(voterLog) = term \wedge LastSubterm(voterLog) > subterm \implies Len(voterLog) > index$, per Lemma 3

        ii. This leads to contradiction following case 10

    (c) $LastTerm(voterLog) > term$. This leads to contradiction following case 11

Having Lemma 4 proved, Lemma R.9 and Theorem R.1 are true, following the same proof in Raft algorithm. Now we have Leader Completeness and State Machine Safety proved for the extended Raft algorithm. □